

Zuno 的 Agentic RAG 与 GraphRAG 理想架构

核心判断

你现在对 Agent 的理解方向是对的：把系统拆成 **LLM、工具、记忆、规划** 四个核心组件，是一个很稳的面试与工程表述。这个拆法和 CoALA 对语言 Agent 的模块化认知架构、Anthropic 对 “workflow 与 agent” 的区分、以及 LangChain/LangGraph 对 agent loop 的工程抽象是相容的。真正需要补上的，不是再发明第五个 “玄学模块”，而是把这四块在工程上继续展开为 **context engineering、hooks/middleware、retrieval router、evidence/trace** 四个支撑层。没有这些层，系统能跑；加上这些层，系统才真正可控、可解释、可评估。 ¹

对 Zuno 而言，最合适的总体方向不是 “一上来就多 Agent”，而是先做成 **单 Agent 主控 + Agentic RAG + GraphRAG + 受控记忆层 + 工具层 + Hooks**。LangChain 官方明确提醒，并不是每个复杂任务都需要 multi-agent；很多场景下，一个拥有合适工具、上下文和动态路由能力的单 Agent 就足够了。Anthropic 也强调，成功率更高的生产实现往往采用 **简单、可组合** 的模式，而不是过度复杂的框架。 ²

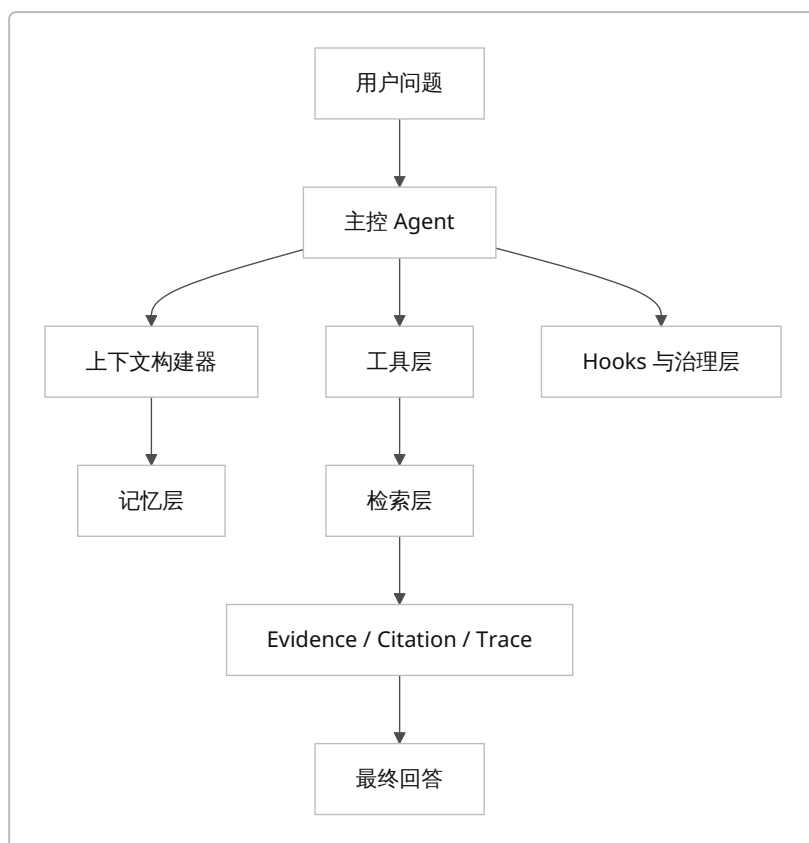
因此，如果你的问题压缩成一句架构结论，我会这样定义 Zuno：

Zuno 应该是一个单 Agent 主控的知识工作台。Agent 负责理解任务、决定是否检索、选择 basic/local/global/drift 路径、并在证据不足时重试；GraphRAG 负责把私有知识从 “chunk 集合” 升级为 “实体—关系—社区—报告” 的结构化底座；记忆层负责把长期偏好、项目事实、会话状态和中间产物组织成可用上下文；工具层与 hooks 则把权限、预算、可观察性和回退机制落到工程上。 ³

四组件如何变成工程架构

你贴出来的笔记把 Agent 节奏概括成 “规划 → 决策 → 执行 → 结果写入记忆 → 再决策”，这个循环本质上就是今天主流 Agent 框架在做的事。LangChain 把 agent loop 直接定义成两步反复执行：**模型调用与工具执行**，直到模型决定结束；ReAct 论文则把它形式化为 reasoning 与 acting 的交替；Deep Agents 则在此基础上补上了执行环境、context management、delegation 和 steering。也就是说，你的四组件理解是对的，但工业实现里会自然长成 “主循环 + 环境 + 约束层” 的样子。 ⁴

从工程视角看，这四个组件在 Zuno 里不该只是概念，而应映射成六层：



这个六层并不是为了“画图好看”，而是因为主流框架都在往这个方向收敛。LangGraph 提供可持久化、可中断、可编排的图式运行时；Deep Agents 把执行环境拆成工具、虚拟文件系统、权限、代码执行和流式事件；LangChain middleware 则提供 before/after model、wrap tool/model 的控制点。你如果把 Zuno 做成“一个 prompt + 一堆工具”，很快就会撞上上下文失控、权限失控、可观察性不足和长任务退化的问题。 5

这也是为什么我不建议你“把工具层、记忆层、钩子层”当成配角。对很多能演示的 Demo 来说，LLM 看起来像系统的大脑；但到了可展示、可面试、可持续演进的项目阶段，真正决定上限的通常是 **上下文工程和运行时治理**。LangChain 官方甚至把“把对的信息、工具和格式在正确时机送进模型”视为 AI 工程师的头号工作。

6

记忆系统设计

Zuno 的记忆层不应该只等于“对话历史 + 向量库”。更好的设计，是把记忆明确分成 **短期状态、工作记忆、长期语义记忆、长期情节记忆、程序性记忆** 五类。LangChain 在 memory 概念文档里明确区分了 short-term 与 long-term memory，并借用了 semantic / episodic / procedural 的分类；CoALA 也把语言 Agent 看成具备模块化记忆和结构化动作空间的系统；MemGPT 进一步提出“多层记忆/分层上下文管理”的操作系统式视角。

7

对 Zuno 来说，最实用的拆法如下：

短期状态 放当前线程内正在进行的事务：本轮用户意图、query_method、最近一次检索结果、当前 plan、工具调用结果、当前 trace id。LangChain 的 short-term memory 是 thread-scoped checkpoint，天然适合承载这一层。 8

工作记忆 放当前任务的中间产物：候选证据包、已打开文件、临时摘要、待验证结论、未完成 todo。Deep Agents 的规划工具、虚拟文件系统、上下文压缩和状态持久化，本质上都在服务这一层。特别是它会把巨大工具结果 offload 到文件系统，并在上下文接近阈值时触发摘要或替换文件指针，这对长任务非常关键。 9

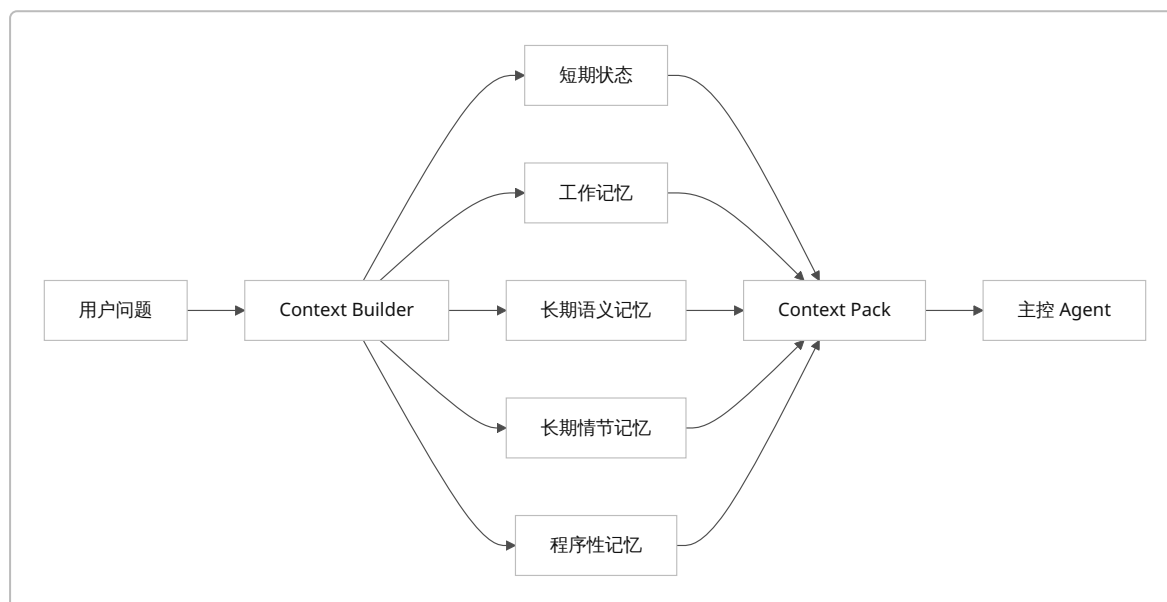
长期语义记忆 放持久事实：用户偏好、项目术语、固定约束、架构基线、模块定义、重要配置、团队约定。LangChain 的 long-term memory 文档说明，长期记忆适合跨线程存储和回忆这类数据，并通过 store 以 JSON 文档形式持久化。Deep Agents 也把 memory 视为始终加载的持久上下文，比如编码风格、偏好、规范和项目指南。 10

长期情节记忆 放过去做过什么：上次为什么选 local 而不是 global、某轮回答为什么失败、哪份文档曾经被判定为高价值来源、某个用户任务最终采用了什么检索策略。CoALA 和 LangChain 都把 episodic memory 定义为对过去行为和经历的记录；Reflexion 则明确展示了如何把“反思文本”写入 episodic buffer，供后续试次改进决策。 11

程序性记忆 放“怎么做事”的规则，而不是事实本身：回答必须区分 Current / Target / Future / History；默认先 basic 再判断是否升级；需要引用时必须要有 evidence；高成本工具触发前先过预算与权限检查。CoALA 把 procedural memory 一部分放在模型权重里，一部分放在 agent code 里；LangChain 也把 procedural memory 类比为 agent 的系统提示与代码规则。 12

Zuno 的关键不是“记住更多”，而是**正确写入、正确检索、正确压缩**。LangChain 文档明确提醒，长上下文会让模型被陈旧内容干扰；Deep Agents 则提供了三类压缩机制：大工具结果下沉到文件系统、大工具输入替换为文件指针、必要时做结构化摘要。换句话说，Zuno 不应把所有历史都塞进 prompt，而应让 Context Builder 生成一个最小但足够的 **Context Pack**。 13

一个适合 Zuno 的记忆总线，可以设计成下面这样：



社区层面的表述也和这个方向一致。牛客网的 Agent 面试类总结，会把短期记忆、长期记忆、Reflection、ReAct、ToT 并列成 Agent 常见构件；一些技术作者在 X 上也倾向于用 episodic / semantic / procedural 的三分法解释 memory。不过这些内容更适合作为“业界通俗表述”，不能替代正式架构依据。 14

规划模块与 Plan、ReAct、Reflection

规划模块是 Zuno 第二个最关键的地方。你的直觉“我们做 Plan + ReAct + Reflection”是合理的，但不应该理解成三套独立框架硬拼在一起，而应该理解成：

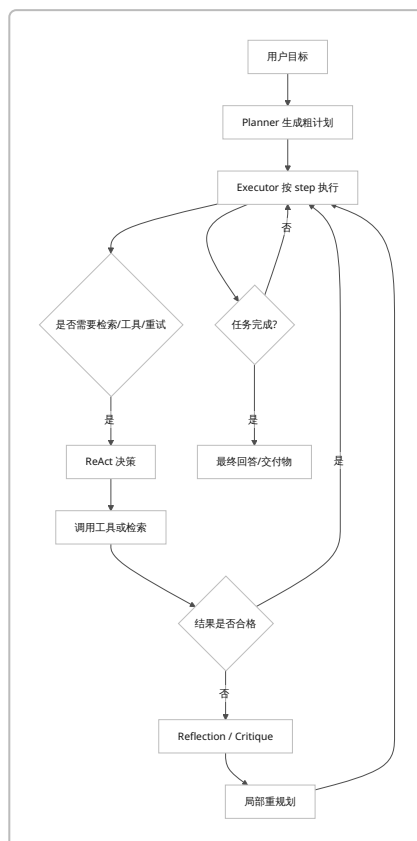
先有一个轻量全局计划，执行时走 ReAct，关键节点再做 Reflection。

这是因为三者各有优缺点。ReAct 的优势是灵活，能边看边改，但它每一步都要重新思考，容易局部最优；Plan-and-Execute 的优势是结构清晰、速度和成本更可控，因为“大模型不必在每一步都重新规划”；Reflection 的优势是能在失败或不确定时回头修正，但若每步都反思，成本会迅速上涨。¹⁵

LangChain 对 plan-and-execute 的总结非常适合拿来当 Zuno 的规划层：它把 planner 与 executor 分离，planner 先生成多步计划，executor 只吃当前 step 和必要上下文；执行后可以触发 re-planning，以决定是否结束或继续。官方还强调，这样做通常比纯 ReAct 更快、更省、更适合多步任务。¹⁶

从论文角度看，显式规划对长程任务是有价值的。Plan-and-Act 这类近期研究延续了这个方向：把高层 Planner 和低层 Executor 分开，有助于处理多步、长时程任务。Reflexion 则给了你 reflection 的工程化方式：不是更新模型权重，而是把语言化反馈写入记忆，用来改善后续决策。¹⁷

所以，Zuno 更适合的规划/执行范式是：



在这个架构里，Planner 不需要细到“点击第几个按钮、调用哪个函数”，它的职责是把问题拆成有限个可检查的工作包。例如：理解问题类型 → 判断是否需要知识库 → 若需要，决定 initial mode/basic or enhanced → 汇总证据 → 生成回答。真正细化到“这一轮要不要查询知识库、走哪个检索器、要不要改写 query”的，是 ReAct 层。Reflection 则只在几个事件触发：证据不足、检索冲突、回答覆盖率低、结构化输出不合格、或者连续两轮没有增量收益。¹⁸

这也是为什么我不建议让 Zuno 默认走 ToT 或树搜索。ToT 和 LATS 这类方法在复杂搜索上有上限优势，但成本显著更高，适合作为“困难模式”或离线分析路径，而不是产品默认交互路径。Reflection 也应是门控反思，不是每步都反思。 ¹⁹

检索与知识增强

你的 retrieval taxonomy 基本已经成型，而且很像一个能落地的 GraphRAG 前端抽象：

- basic = BM25 + Dense Vector
- local = Basic + Entity Linking + Graph Neighborhood + Chunk Backlink
- global = Community Reports + Map-Reduce Summary + Supporting Evidence
- drift = Community Primer + Follow-up Questions + Local/Basic Retrieval + Reduce
- auto = Router 自动选择

这套分法与 Microsoft GraphRAG 的官方查询引擎是对齐的。GraphRAG 官方明确把 Query Engine 拆成 Local Search、Global Search、DRIFT Search 和 Basic Search。Basic 本质上是简单 vector RAG；Local 会以相关实体为入口，拉取 entity、relationship、community report、text units 等；Global 会基于 community reports 做 map-reduce；DRIFT 则把 community information 引入 local search，通过 top-K 社区报告生成初始回答与 follow-up questions，再往局部细化。 ²⁰

GraphRAG 的索引侧也和你的想法一致：它先把文档切成 TextUnit，再抽 Entity、Relationship、Covariate/Claim，做 Hierarchical Leiden 社区检测，最后生成 Community Report 并对 text units / entities / community reports 做 embedding。也就是说，GraphRAG 不是不要向量检索，而是把向量检索建立在结构化产物之上。 ²¹

这里最关键的问题，是你问到的：

global 能不能和 BM25 + RAG 组合？

答案是：能，但不适合直接在“原始 chunk 排名”这一层做硬融合；更适合在“检索计划层”或“综合生成层”做组合。GraphRAG Global Search 官方流程用的是 community reports 的 map-reduce，不是原始 chunk 排序；而 basic / local 往往输出 chunk、entity、relationship 等细粒度证据。因此，把 global 结果和 BM25 top-k 直接丢进同一个 RRF 排行里并不自然。更合理的做法是把 global 作为 **primer / corpus-level prior**，再用 basic 或 local 去做证据回补和引用固化；这其实就是 DRIFT 的核心思路。 ²²

从研究角度看，把向量 RAG 与 GraphRAG 混合是可行的，而且确实有人这么做。近期对 Vector RAG、GraphRAG 与 Hybrid GraphRAG 的对比研究显示，混合式方案能同时利用语义覆盖与结构化关系，常在事实正确性上占优，但代价是冗余和计算成本更高。另一项系统评估也指出，RAG 与 GraphRAG 各有强项，值得研究集成策略，而不是简单替代。 ²³

基于这些来源，我会给 Zuno 一个非常清晰、产品和底层分离的设计：

产品层保留三个模式。

- **普通模式**：只能走 basic。适合低延迟、可预期问答。
- **增强模式**：一定走受控 Agentic RAG，但 query_method 由系统在 basic/local/global/drift 中自动选择。
- **自动模式**：先由 Router 判断 direct / basic / enhanced，再进入对应链路。

这种设计的好处是，用户有可解释的显式选择，系统也保留自动优化空间。LangChain 官方对 retrieval 的分类正好支持这种分层：2-step RAG 适合快而稳的场景，Agentic RAG 适合由模型决定何时检索，Hybrid RAG 适合在中间加入 query enhancement、retrieval validation 与 answer validation。Zuno 的“增强模式”本质上就是 **Hybrid + Agentic RAG**。 ²⁴

底层保留四个 query methods。

这是给系统内部路由器和 trace 用的，而不是直接暴露所有技术细节给普通用户。用户看到的是“普通 / 增强 / 自动”；系统内部记录的是 `query_method = basic | local | global | drift`。这样一来，产品层简单，日志层清晰，评测层也更容易做切片分析。 ²⁵

每个 method 不必都是“同样形式的多路召回”。

`basic` 适合做 BM25 + dense + RRF/Rerank；`local` 适合在 basic 上叠实体邻域和 chunk backlink；`global` 更像 summary retriever / report synthesizer，不一定走 RRF；`drift` 更像一个两阶段搜索器，先 global primer 再 local/basic evidence loop。Azure AI Search 与多家检索系统文档都表明，BM25 + vector 的混合检索通常通过 RRF 这类 rank fusion 合并结果；但对于 global 这种 map-reduce 社区摘要路径，更自然的融合位置是在“后续 evidence merge 与 response synthesis”。 ²⁶

因此，Zuno 最合理的 query router 可以这样理解：

- 问“某个字段/模块在哪里”时，先 basic。
- 问“某个实体和另一个模块是什么关系”时，走 local。
- 问“整个项目的主要架构/技术债/主题”时，走 global。
- 问“迁移风险、架构诊断、优先级建议”时，走 drift。
- 如果用户没指定，就由 Router 判断；如果用户指定了，则优先尊重用户。 ²⁷

最后，Agentic RAG 的时机也应该明确：**不是“先 RAG 再让 Agent 总结”，而是 Agent 在每一轮 ReAct 中决定这一轮是否需要查询知识库**。LangGraph 官方 agentic RAG 教程就是这么做的：先决定是否 retrieval，再 grade documents，不够就 rewrite question，够了再 answer。这个思路和你想要的“Agent 自主决定本轮要不要开知识库、要不要补 local/global/drift”完全一致。 ²⁸

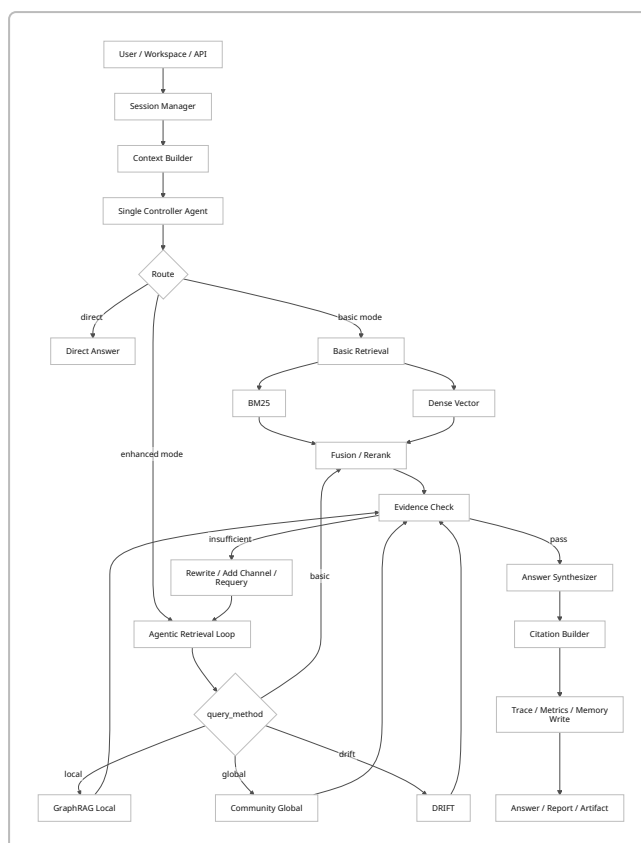
开源案例与 Zuno 的目标架构

从成熟开源实现看，你可以同时借鉴两类项目，但不要照抄。

第一类是 **Deep Agents / Open Deep Research** 这种“单主控、强运行时”的方向。Deep Agents 把代理能力组织成 execution environment、context management、delegation 和 steering，并内建 task planning、subagent、memory、filesystem、human-in-the-loop 等能力；`write_todos` 这种轻量规划工具尤其适合你的长任务工作台场景。Open Deep Research 则展示了一个成熟的研究 Agent 产品壳：它是一个简单、可配置的开源 deep research agent，支持不同模型、搜索工具和 MCP server；它的 README 还保留了两种 historical 架构思路——一个偏 workflow 的 plan-and-execute with reflection，另一个偏 multi-agent 的 supervisor-researcher。这个对你很有启发：**成熟项目并不是一开始就选定唯一架构，而是逐渐收敛到最可维护的默认形态**。 ²⁹

第二类是 **didilili/deepsearch-agents** 这种“多来源研究助手”的方向。它采用典型的 orchestrator-workers 模式，由主智能体调度网络搜索助手、数据库助手和 RAGFlow 助手，配合同步文件工具、WebSocket 事件流与 `thread_id` / `session_dir` / `ContextVar` 的会话隔离机制。这个项目最值得你借鉴的不是“多 Agent”本身，而是它已经把 **会话隔离、工具分工、可观察执行链路、前后端闭环** 展示得非常清楚。对 README 展示来说，这是非常好的模板。 ³⁰

综合官方文档、论文与这些开源项目，我建议 Zuno 的**目标架构**设计成下面这个形态：



这张图里最核心的部分有三条。

第一条：单 Agent 主控，而不是默认多 Agent。

这是因为官方和业界都在提醒，过早进入 multi-agent 会把上下文管理、调试、权限边界和失败模式复杂度迅速放大。你完全可以先做一个很强的 controller agent，让 GraphRAG、RAG、MCP、文件系统、评测器都成为它的工具。等到工具数量过多，或者任务必须并行，或者不同子任务需要明显上下文隔离时，再把某些能力提升为 subagent。 ³¹

第二条：中间一定要有 Context Builder。

你的系统不是“用户问题直接丢给 Agent”，而是“先把系统规则、会话状态、长期记忆、候选证据、用户设置拼成一个 Context Pack，再交给 Agent”。LangChain 把 context engineering 视为 agent 可靠性的命门；Deep Agents 的上下文压缩、技能加载、内存与文件系统，本质上都在服务这一步。 ³²

第三条：必须有 Hooks / Middleware / Evidence Check。

LangChain middleware 提供了 before_agent、before_model、after_model、wrap_model_call、wrap_tool_call 等执行点，可以用来做权限、预算、PII、防错、动态工具选择、fallback 与日志。对 Zuno 来说，你至少应该有四类 hook：

一类在 agent 开始时注入 thread metadata 和 route policy；一类在 model call 之前裁剪上下文和限制工具面；一类在 tool call 前后做权限、缓存、计量和结果标准化；一类在最终输出前做 citation coverage 和 evidence coverage 检查。 ³³

把这些合起来，Zuno 的 README / 展示文档应该尽量强调的，不是“我们支持很多最热词”，而是下面这几个明确卖点：

- **普通模式 vs 增强模式 vs 自动模式**：让用户能理解产品行为。
- **basic/local/global/drift**：让面试官能理解你的技术路由。

- **Evidence / Citation / Trace**：让系统可核查、可调试。
- **Memory / Tool / Hook**：让项目看起来像一个真正的 agent runtime，而不是聊天机器人拼装。
- **单 Agent 先行，必要时再演进 subagent**：让架构选择显得克制而成熟。 ³⁴

最适合 Zuno 的落地建议

如果把上面的研究最终收敛成一个可执行的产品与工程决策，我会建议你这样定稿。

产品层三模式保留。

保留“普通模式 / 增强模式 / 自动模式”是对的。普通模式只允许 basic；增强模式由系统自动在 basic/local/global/drift 中选；自动模式则让 Router 先判断 direct/basic/enhanced，再进入相应策略。这样既满足可控性，也满足智能性。 ³⁵

底层四方法保留。

basic/local/global/drift 的区分应继续保留，而且要写进 trace。原因很简单：这不仅决定系统行为，也决定评测维度。GraphRAG 官方已经把这四条路径定义得很清楚；你要做的是把它们封装成 Zuno 内部的 query_method，而不是把复杂性直接甩给产品用户。 ²⁵

global 不和 BM25 生硬拼榜单，而是做“先总后细”的两段式。

如果用户问的是全局性问题，就让 global 先产出 corpus-level prior，再用 local/basic 去追关键证据；如果问题本身就是复杂探索型，直接走 drift，因为 DRIFT 官方定义的就是 global + local 的结合。你若想再进一步，可以在 response synthesis 阶段把 global 的关键 points 变成“候选结论”，再由 basic/local 检查每条结论是否有 chunk-level 支撑。这样既兼顾 community reports 的全局视野，也保留 citation 的细粒度基础。这个组合方式不是 Microsoft 文档逐字给出的“唯一标准答案”，但它与官方 query flows 和现有 hybrid GraphRAG 研究是一致的工程推论。 ³⁶

规划范式采用“粗计划 + ReAct 执行 + 门控 Reflection”。

不要在每个用户问题上都重型规划；也不要完全无计划地全靠 ReAct 乱走。最省心的默认，是先生成一个粗粒度 todo 或 step list，再逐步执行；失败、冲突或不确定时，再触发 reflection 与局部 replanning。Deep Agents 自带 write_todos 这种轻量计划工具，其实就很接近你需要的感觉。 ³⁷

记忆层按“短期 + 工作 + 语义 + 情节 + 程序”做。

这是 Zuno 与普通 RAG 产品的重要差异点。你的系统不只是“查资料回答”，而是“会持续工作、生成中间产物、记住偏好、沉淀经验、遵守规则”。这时，单一向量库式 memory 已经不够了。要把长期事实、运行轨迹和策略规则分开存。 ³⁸

工具层与 MCP 必须标准化。

把 BM25/vector/graph searches、文件读写、评测、数据接口、外部服务都做成 Tool Registry 下的统一 contract；外部连接优先走 MCP。Anthropic 与 MCP 官方文档都把 MCP 定义为连接外部系统的开放标准；Deep Agents 也原生支持从 MCP server 加载工具。对 Zuno 来说，这意味着未来接 IDE、数据库、网页、企业 API、甚至调试器，都不必为每个系统重新发明工具协议。 ³⁹

Hooks 必须成为架构的一部分，而不是事后补丁。

在 Zuno 里，hooks 不只是 logging。它们应该负责：动态裁剪上下文、限制工具面、批准敏感调用、记账 latency 和 token、执行 fallback、把 route/evidence/citation 写入 trace，以及决定哪些结果允许进入长期记忆。只有这样，Zuno 才能从“会跑的 agent demo”升级为“有治理能力的知识系统”。 ³³

归根结底，最适合你的目标架构不是“再加更多 buzzword”，而是把一句话真正做实：

Zuno 不是一个先检索再回答的 RAG 聊天框，而是一个以单 Agent 为主控、以 Agentic RAG 为控制层、以 GraphRAG 为结构化知识底座、以 Memory / Tools / Hooks 为运行时支撑、以 Evidence / Citation / Trace 为可信输出的知识工作台。 40

- 1 Cognitive Architectures for Language Agents
https://arxiv.org/abs/2309.02427?utm_source=chatgpt.com
- 2 31 Multi-agent - Docs by LangChain
<https://docs.langchain.com/oss/python/langchain/multi-agent>
- 3 28 40 Build a custom RAG agent with LangGraph - Docs by LangChain
<https://docs.langchain.com/oss/python/langgraph/agent-ric-rag>
- 4 6 32 Context engineering in agents - Docs by LangChain
<https://docs.langchain.com/oss/python/langchain/context-engineering>
- 5 LangGraph overview - Docs by LangChain
https://docs.langchain.com/oss/python/langgraph/overview?utm_source=chatgpt.com
- 7 8 13 Short-term memory - Docs by LangChain
<https://docs.langchain.com/oss/python/langchain/short-term-memory>
- 9 29 37 Deep Agents overview - Docs by LangChain
<https://docs.langchain.com/oss/python/deepagents/overview>
- 10 Long-term memory - Docs by LangChain
<https://docs.langchain.com/oss/python/langchain/long-term-memory>
- 11 38 Memory overview - Docs by LangChain
<https://docs.langchain.com/oss/python/concepts/memory>
- 12 Cognitive Architectures for Language Agents
https://arxiv.org/html/2309.02427v3?utm_source=chatgpt.com
- 14 AI Agent框架（LLM Agent）：LLM驱动的智能_牛客网
<https://www.nowcoder.com/discuss/638693934130085888>
- 15 18 ReAct: Synergizing Reasoning and Acting in Language Models
https://arxiv.org/abs/2210.03629?utm_source=chatgpt.com
- 16 Plan-and-Execute Agents
<https://www.langchain.com/blog/planning-agents>
- 17 Plan-and-Act: Improving Planning of Agents for Long-Horizon Tasks
https://arxiv.org/abs/2503.09572?utm_source=chatgpt.com
- 19 Tree of Thoughts: Deliberate Problem Solving with Large Language Models
https://arxiv.org/abs/2305.10601?utm_source=chatgpt.com
- 20 25 Overview - GraphRAG
<https://microsoft.github.io/graphrag/query/overview/>
- 21 Dataflow - GraphRAG
https://microsoft.github.io/graphrag/index/default_dataflow/
- 22 36 Global Search - GraphRAG
https://microsoft.github.io/graphrag/query/global_search/

23 Benchmarking Vector, Graph and Hybrid Retrieval Augmented Generation (RAG) Pipelines for Open Radio Access Networks (ORAN)

<https://arxiv.org/html/2507.03608v1>

24 34 35 Retrieval - Docs by LangChain

https://docs.langchain.com/oss/python/langchain/retrieval?utm_source=chatgpt.com

26 Hybrid Search Scoring (RRF) - Azure AI Search

https://learn.microsoft.com/en-us/azure/search/hybrid-search-ranking?utm_source=chatgpt.com

27 Welcome - GraphRAG

<https://microsoft.github.io/graphrag/>

30 deepsearch-agents/README.md at main • didilili/deepsearch-agents • GitHub

<https://github.com/didilili/deepsearch-agents/blob/main/README.md>

33 Custom middleware - Docs by LangChain

https://docs.langchain.com/oss/python/langchain/middleware/custom?utm_source=chatgpt.com

39 Introducing the Model Context Protocol

https://www.anthropic.com/news/model-context-protocol?utm_source=chatgpt.com